

# Indexes

## Why we need indexes

---

Indexes help quickly locate data without looking for it everywhere.

- Example: In the British Library, find *Designing Data-Intensive Applications* by Martin Kleppmann
  - **Without indexes:** You will need to look through all of the books
  - **With indexes:** Go to Computers / Databases / M (for Martin)



- Indexes help quickly locate data without looking for it everywhere. For example, the British Library holds 25 million books. You are looking for 'Designing Data-Intensive Applications' by Martin Kleppmann. Without indexes, you will need to look through all of them. With indexes, you go into the Computers section, then into the Databases aisle, and finally on the M shelf. There you will find the book by Martin Kleppmann.

## Everyday index samples

---

- Telephone book
- Dictionary
- Index at the end of a book



- We use indexes every day to find information quickly. For example: Until just a few years ago, telephone books were really popular. All the names were indexed by surname and first name. So, to find one person, you jump into their surname section. D for Doe, then you go to first names starting with J , and you can easily find John Doe. Other everyday examples include a dictionary and an index at the end of book.

## When to index

---

- Most frequent queries

```
db.courseEnrollment.find({ "courseId": 1547 })
```

```
db.courseEnrollment.createIndex({ "courseId": 1 })
```

- In database systems, indexes should be created for the most frequent queries. In the Campus Management Database, for the course enrollment collection, we find students using 'courseId'. Instead of scanning the whole collection, we create an index on the field 'courseId' in the course enrollment collection. 'courseId : 1' means store the index in ascending order. Our index on 'courseId' will help find those documents more efficiently.

## Use indexes to sort

---

Indexes can help with sorting

```
db.courseEnrollment.find({ "courseId": 1547 }).sort({ "studentId": 1 })  
db.courseEnrollment.createIndex({ "courseId": 1, "studentId": 1 })
```

- But to organize the students by 'studentId' in ascending order, MongoDB will need to perform in-memory sort, which will not be efficient. In this situation, we can change our index to be a compound index, which means indexing more than one field. Because the items in the index are organized in ascending order, once we find all documents with a matching 'courseId' of '1547', they will all be already sorted by 'studentId' because of this index.

## Indexes in MongoDB

---

- MongoDB indexes are special data structures
- They store the fields you are indexing
- They also store the location of the document

- Indexes in MongoDB are special data structures. They store the fields you are indexing, and a location where that document is stored on disk.

## How MongoDB stores indexes

---

```
db.courseEnrollment.createIndex({ "courseId": 1, "studentId": 1 })
```

- MongoDB stores indexes in a tree form: a balanced tree to be precise. Imagine our last example: 'courseId,' being the first field in our compound index, is in ascending order. Under each tree node, you will have students in ascending order too.

# How MongoDB stores indexes

```
db.courseEnrollment.createIndex({ "courseId": 1, "studentId": 1 })
```



- This makes finding documents much more efficient, whether it's an equality or a range search. And if you are sorting on a field which is already indexed, MongoDB doesn't need to sort it again.

## What would happen if we didn't use indexes?

If we didn't use indexes in a database, searching for information would be much slower and less efficient. Here are a couple of key points to consider:

- **Full Table Scans:** Without indexes, the database would have to look through every single record in a table to find the information you need. This is called a full table scan. For large databases, this can take a lot of time and resources.
- **Increased Load:** Searching without indexes can put a heavy load on the database server, as it has to process many more records. This can lead to slower performance for all users, especially if multiple queries are being run at the same time.

In summary, not using indexes can lead to slower searches and increased strain on the database, making it less efficient overall.

## How can indexes improve database performance practically?

Indexes can significantly improve database performance in several practical ways:

- **Faster Query Execution:** When you create an index on a field that is frequently searched, the database can quickly locate the relevant records without scanning the entire table. This reduces the time it takes to execute queries.
- **Efficient Sorting:** If you have an index on a field that you often sort by, the database can retrieve the sorted data directly from the index. This eliminates the need for additional sorting operations, which can be time-consuming.
- **Reduced Resource Usage:** By minimizing the number of records the database needs to scan, indexes help reduce CPU and memory usage. This leads to better overall performance, especially in high-traffic applications.
- **Improved Join Operations:** When joining tables, indexes can speed up the process by allowing the database to quickly find matching records in both tables, rather than searching through all records.

In summary, indexes enhance database performance by speeding up searches, sorting, and join operations while reducing resource consumption.